<u>README EN ESPAÑOL</u>

Algoritmo de clasificación de expresiones de odio en mensajes en español. Este algoritmo fue desarrollado en el marco del proyecto Hatemedia (PID2020-114584GB-I00), financiado por MCIN/AEI /10.13039/501100011033.

<u>Importación y uso del modelo para realizar predicciones:</u>

Los diferentes modelos empleados tienen como fin analizar datos extraídos de diferentes redes sociales como por ejemplo Twitter o Facebook de la cual se extraen los comentarios referentes a los periódicos (La Vanguardia, El Mundo, ABC, El País y 20 Minutos) y páginas web de estos para su posterior clasificación en función de si estos contienen o no odio.

Para llevar a cabo esta clasificación, se han entrenado distintos modelos como *Gradient Boosting*, SVM (*Support Vector Machines*), RF (*Random Forest*) o CART (*Classification And Regression Tree*).

Para el uso de los diferentes modelos de predicción se adjuntan los scripts Python "ejemplo.py" y "obtencion_caracteristicas.py" y la carpeta "recursos" en la que se encuentran las distintas bolsas de palabras utilizadas para la obtención de características en el preprocesado de los comentarios a predecir.

Antes de ejecutar estos scripts es necesario instalar las distintas librerías importadas en ellos.

OBTENCION_CARACTERISTICAS.PY:

Las funciones presentes en este script son necesarias para el preprocesado del corpus. Los distintos modelos necesitan las características extraídas mediante este script para realizar sus predicciones.

EJEMPLO.PY:

Se detalla a continuación mediante el código de este script el procedimiento a seguir para realizar las predicciones.

Importamos la librería joblib para poder cargar los distintos modelos, así como importamos el otro script adjunto y la librería panda para trabajar con dataframes:

```
import joblib
import obtener_caracteristicas
import pandas as pd
```

Cargamos el modelo mediante el uso de la librería joblib:

```
# Cargar el modelo
modelo = joblib.load("nombre_del_modelo.joblib")
```

Leemos el archivo csv en el que están presentes los distintos comentarios sobre los que queremos realizar las predicciones. Para que el preprocesado se lleve a cabo correctamente necesitamos nombrar como “CONTENIDO A ANALIZAR” la columna que contiene los comentarios.

Se crea un dataframe que contendrá los comentarios, al que más tarde se añadirán las predicciones.

```
# Cargamos el archivo sobre el que realizar las predicciones
df = pd.read_csv("archivo_a_predecir.csv", delimiter=';')

# Dataframe en el que mostrar los resultados
resultados = pd.DataFrame()
resultados['CONTENIDO A ANALIZAR'] = df['CONTENIDO A ANALIZAR']
```

Se obtienen las características que utilizará el modelo para realizar la predicción de llos comentarios:

```
# Procesamos el dataframe
df = obtener_caracteristicas.main(df)
```

Obtenemos las predicciones y mostramos los resultados:

```
# Realizar la predicción
prediccion = modelo.predict(df)

# Añadimos las predcciones para visualizar los resultados
resultados['PREDICCIONES'] = prediccion
print(resultados)
```

Uso del código de entrenamiento:

Se adjunta el notebook de Google Colab “Modelo_binario_.ipynb”, siendo aquí donde encontramos el código a ejecutar para el entrenamiento de los modelos.

Colab es un producto de Google Research que permite a cualquier usuario escribir y ejecutar código arbitrario de Python en el navegador; es especialmente adecuado para tareas de aprendizaje automático, análisis de datos y educación. Con esta herramienta veremos el código en distintas celdas que podremos ejecutar de manera independiente.

Al lanzar la siguiente celda se importan las distintas librerías necesarias para la ejecución de todo el código:

```python
import pandas as pd
import numpy as np
import tensorflow as tf
import tensorflow.keras as kr
import keras
import matplotlib.pyplot as plt
import joblib
import warnings

# Modelos
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.naive_bayes import BernoulliNB
from keras.models import Sequential
from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import (
    train_test_split,
    cross_val_score,
    RepeatedKFold,
    GridSearchCV,
    RandomizedSearchCV
)

from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    ConfusionMatrixDisplay,
    precision_score,
    recall_score,
    f1_score
)
from keras.layers import Dropout
from keras.layers import Dense
from keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.metrics import AUC
from keras.models import load_model
from keras.utils import np_utils
from tqdm import tqdm
warnings.filterwarnings('ignore')
```

Si es la primera vez que ejecutamos esta celda, antes debemos instalar las distintas librerías en el entorno de ejecución que proporciona Google Colab, de la siguiente forma:

! pip install nombre_de_la_libreria

En las siguientes celdas se realiza lo siguiente:

1. La función get_X_y(df) retorna por separado las características y la etiqueta asociada (odio = 1, no odio = 0) con las que se entrena y valida el modelo.

```
def get_X_y(df):
    FEATURES = ['A_t','B_t','C_t','D_t','E_t','Valor_1','Valor_2','Valor_3',
                'Valor_4','Valor_5','Valor_6','Valor_7','Valor_8','Valor_9','Valor_10']
    TARGET = 'label'
    #TARGET = 'INTENSIDAD'
    X = df[FEATURES]
    y = df[TARGET]

    return X, y
```

2. Leemos el archivo .csv a un DataFrame de pandas.

```
df_prueba = pd.read_csv("Ruta_Dataset/tu_archivo.csv", delimiter=";")
```

3. Fusionar intensidades ejecutando

```
# Fusionar intensidades si es necesario
if len(df_prueba['label'].value_counts()) != 2:
    df_prueba['label'] = df_prueba['label'].apply(lambda x: 1 if x==2 else (x))
    df_prueba['label'] = df_prueba['label'].apply(lambda x: 1 if x==3 else (x))
    df_prueba['label'] = df_prueba['label'].apply(lambda x: 1 if x==4 else (x))
    df_prueba['label'] = df_prueba['label'].apply(lambda x: 1 if x==5 else (x))
    df_prueba['label'] = df_prueba['label'].apply(lambda x: 1 if x==6 else (x))
```

Empezamos preparando el dataset separando las entradas que contienen odio y las que no.

```
# Separar las intensidades 1 y 0 (odio / no odio)

intensidad_0_df_bucle = df_prueba[df_prueba['label'] == 0.0]
intensidad_1_df_bucle = df_prueba[df_prueba['label'] == 1.0]

rows_count_i0 = len(intensidad_0_df_bucle.index)
rows_count_i1 = len(intensidad_1_df_bucle.index)

list_row_count = [
    rows_count_i0,
    rows_count_i1,
]
print(f'Total: {len(df_prueba.index)}')
print(f'Intensidad 0: {rows_count_i0}')
print(f'Intensidad 1: {rows_count_i1}')
```

Ejecutamos para obtener las secciones del dataset que consigan un mayor accuracy para posteriormente realizar el entrenamiento de los modelos.

```
# Bucle para obtener el mejor 'trozo' del dataset de no odio, aqui se hacen entrenamientos
# independientes, como se ha explicado en el informe 1
inicio = 0
minimo = min(list_row_count)
accuracy_list_gb = []
metrics_list = []
for i in tqdm(range(inicio, rows_count_i0, minimo)):
    if i == 0:
        i = minimo
    if inicio != i:
        intensidad_0_df_total = df_prueba[df_prueba['label'] == 0.0]
        intensidad_0_df = intensidad_0_df_total.iloc[inicio:i]
        balanced_df = pd.concat([intensidad_0_df, intensidad_1_df_bucle])

        X, y = get_X_y(balanced_df)
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1)

        gradient = GradientBoostingClassifier(
            n_estimators = 100,
            max_depth = 3,
            learning_rate = 0.1,
            max_features = 'sqrt'
         )

        gradient.fit(X_train, y_train)
        pred = gradient.predict(X_test)
        acc_score = accuracy_score(pred, y_test)

        precision_micro = precision_score(y_test, pred, average='binary')
        #print(f'Precision micro: ' "{0:.4f}".format(precision_micro))

        recall_micro = recall_score(y_test, pred, average='binary')
        #print(f'Recall micro: ' "{0:.4f}".format(recall_micro))

        f1_micro = f1_score(y_test, pred, average='binary')
        #print(f'F1 micro: ' "{0:.4f}".format(f1_micro))

        metrics_list.append({
            'Inicio':inicio,
            'Fin':i,
            'Precision':precision_micro,
            'Recall':recall_micro,
            'F1':f1_micro,
            'Accuracy':acc_score
        })

        if acc_score >= 0.85:
            accuracy_list_gb.append({
                'Inicio':inicio,
                'Fin':i,
                'Accuracy':acc_score
            })

    inicio = i
```

Obtenemos la sección con la que se ha conseguido el mayor accuracy para conformar el cuerpo de entrenamiento y validación de los distintos modelos.

```
# Recorrer la lista de diccionarios y guardar los valores de Accuracy en una lista
# (transformar lista de diccionarios en list solo de accuracy)
lista_valores_acc = [item['Accuracy'] for item in accuracy_list_gb]
lista_valores_acc
```

```
# Obtener valores Inicio y Fin en función de valor máximo de Accuracy
maximo = max(lista_valores_acc)
inicio_entrenar = 0
fin_entrenar = 0
for diccionario in accuracy_list_gb:
    if diccionario['Accuracy'] == maximo:
        inicio_entrenar = diccionario['Inicio']
        fin_entrenar = diccionario['Fin']
print(f"Para el valor de Accuracy {maximo}, Inicio es {inicio_entrenar} y Fin es {fin_entrenar}.")
```

```
# Obtener el trozo del dataset de no odio donde mayor accuracy se haya obtenido, unirlo con el de odio
# y separar en X e Y para entrenar los modelos solo con esa parte.
intensidad_0_df_total = df_prueba[df_prueba['label'] == 0.0]
intensidad_0_df = intensidad_0_df_total.iloc[inicio_entrenar:fin_entrenar]
balanced_df = pd.concat([intensidad_0_df, intensidad_1_df_bucle])

X, y = get_X_y(balanced_df)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1)
```

Las siguientes celdas servirán para lanzar el entrenamiento, en este caso, del modelo *Gradient Boosting* (a continuación aparecen las celdas con el código correspondiente al resto de modelos); y visualizar los resultados en el reporte de clasificación y la matriz de confusión pintada con la librería matplotlib.

```
[ ] # Entrenar modelo con valores Inicio y Fin según el mayor Accuracy

    gradient = GradientBoostingClassifier(
        n_estimators = 100,
        max_depth = 3,
        learning_rate = 0.1,
        max_features = 'sqrt'
    )

    gradient.fit(X_train, y_train)
    pred = gradient.predict(X_test)
    acc_score = accuracy_score(pred, y_test)
    precision = classification_report(y_test, pred)

    print('Accuracy:',acc_score)

    recall_micro = recall_score(y_test, pred, average='micro')
    print(f'Recall micro: ' "{0:.4f}".format(recall_micro))

    precision_micro = precision_score(y_test, pred, average='micro')
    print(f'Precision micro: ' "{0:.4f}".format(precision_micro))

    f1_micro = f1_score(y_test, pred, average='micro')
    print(f'F1 micro: ' "{0:.4f}".format(f1_micro))

    print(precision)
```

```
[ ] labels = [0,1]
    GradientBoosting = confusion_matrix(y_true = y_test, y_pred = pred, normalize='true', labels = labels)

    confusionMatrixDisplay = ConfusionMatrixDisplay(
        confusion_matrix = GradientBoosting
    )

    confusionMatrixDisplay.plot(cmap='Blues')
    plt.show()
```

En una última celda podemos descomentar la segunda línea si queremos exportar un modelo ya entrenado.

```
# Guardar modelo si se necesita
#joblib.dump(gradient, 'modelos_guardados/GradientBoosting_binario.pkl')
```

Authores:

- Elias Said-Hung
- Julio Montero-Díaz
- Oscar De Gregorio
- Almudena Ruiz
- Xiomara Blanco
- Juan José Cubillas
- Daniel Pérez Palau

Financiado por:

Agencia Estatal de Investigación – Ministerio de Ciencia e Innovación

Con el apoyo de:

- POSSIBLE S.L.

Más información

- https://www.hatemedia.es/ o contactar con: elias.said@unir.net